Flight Booking System

Software Engineering Project Final Report

README.md Title Page

Field	Details
Group Number	Group 09
Case Study	Flight Booking System
Course	Software Engineering - Winter 2026
Instructor	Samer ElKababji

Team Members

Name	Student ID
Elias Hreish	20220677
Ghaith Hajarat	20220445
Faris Samara	20220553
Ahmad Abed	20220976

Overview

A web-based and database-driven platform that enables users to search, book, and manage flight reservations. Customers can browse available flights, select seats, make online payments, and receive electronic tickets. Airline administrators manage flight schedules and seat availability, while system administrators monitor users and system settings.

Tools Used

- PlantUML - UML and C4 diagram source files
- Draw.io - Supplementary diagram editing
- Visual Studio Code - Primary IDE
- Git / GitHub - Version control and collaboration
- Pandoc - Markdown to PDF conversion

Diagrams

Diagram	File	Description
C4 Level 1 - Context	uml/c4_context.puml	Shows the system boundary and external actors/systems
C4 Level 2 - Container	uml/c4_container.puml	Shows internal containers such as web app, API server, database, and services
Activity Diagram with Swimlanes	uml/activity_swimlane.puml	Shows the booking workflow across Customer, System, and Payment Gateway
Use Case Diagram	uml/use_case.puml	Shows system use cases for Customer, Airline Admin, and System Admin
High-Level Sequence Diagram	uml/sequence_highlevel.puml	Shows stakeholder-level flight booking interaction
Detailed Sequence Diagram	uml/sequence_detailed.puml	Shows developer-level API, database, payment, and notification flow
Class Diagram	uml/class_diagram.puml	Shows system classes, attributes, operations, associations, inheritance, aggregation, and composition
DFD Level 1	uml/dfd_level1.puml	Shows data flow between users, processes, data stores, and external systems

Repo Structure

The repository is organized into the following main folders and files:

Item	Explanation
README.md	Project title page with group information, overview, diagrams, repo structure, and contributions.
docs/	Contains the Markdown report and final PDF report.
uml/	Contains PlantUML source files and exported PNG diagrams for C4, UML, class, sequence, activity, and DFD models.

Contributions

Member	Role	Commits
Elias Hreish	C4 context diagram, C4 container diagram, and architecture review	1
Ghaith Hajarat	Use case diagram, use case descriptions, and interaction review	1
Faris Samara	High-level sequence diagram, detailed sequence diagram, and final report organization	1
Ahmad Abed	Activity diagram, class diagram, DFD, README, and final documentation review	1

1. System Description

The Flight Booking System is a web-based and database-driven platform that enables customers to search, book, and manage flight reservations online. The system connects customers, airline administrators, system administrators, payment services, notification providers, and airline data sources in one unified booking environment.

Customers use the system to search available flights by origin, destination, date, and passenger count. After selecting a flight, the customer can choose seats, enter passenger information, review the booking summary, complete online payment, and receive an electronic ticket. Airline administrators use the system to manage flight schedules, update seat availability, and review booking statistics. System administrators manage user accounts, monitor system behavior, and configure platform settings.

The system stores users, passengers, flights, seat maps, bookings, payments, e-tickets, notifications, and booking history in a central database. It integrates with an external airline database to keep flight schedules and seat availability updated, a payment gateway to process transactions and refunds, and an email/SMS provider to deliver booking confirmations and e-tickets.

Main goals of the system are:

- Allow customers to book flights online in a secure and simple flow.
- Maintain accurate flight, seat, booking, and payment records.
- Support airline administrators with schedule and seat management.
- Support system administrators with monitoring and user management.
- Provide reliable notifications and electronic tickets after successful booking.

2. C4 Model Level 1 - Context Diagram

The context diagram shows the Flight Booking System as one main software system and identifies the external people and external systems that interact with it.

Main actors: Customer, Airline Administrator, and System Administrator.

External systems: Payment Gateway, Notification Service / Email-SMS Provider, and Airline Database.

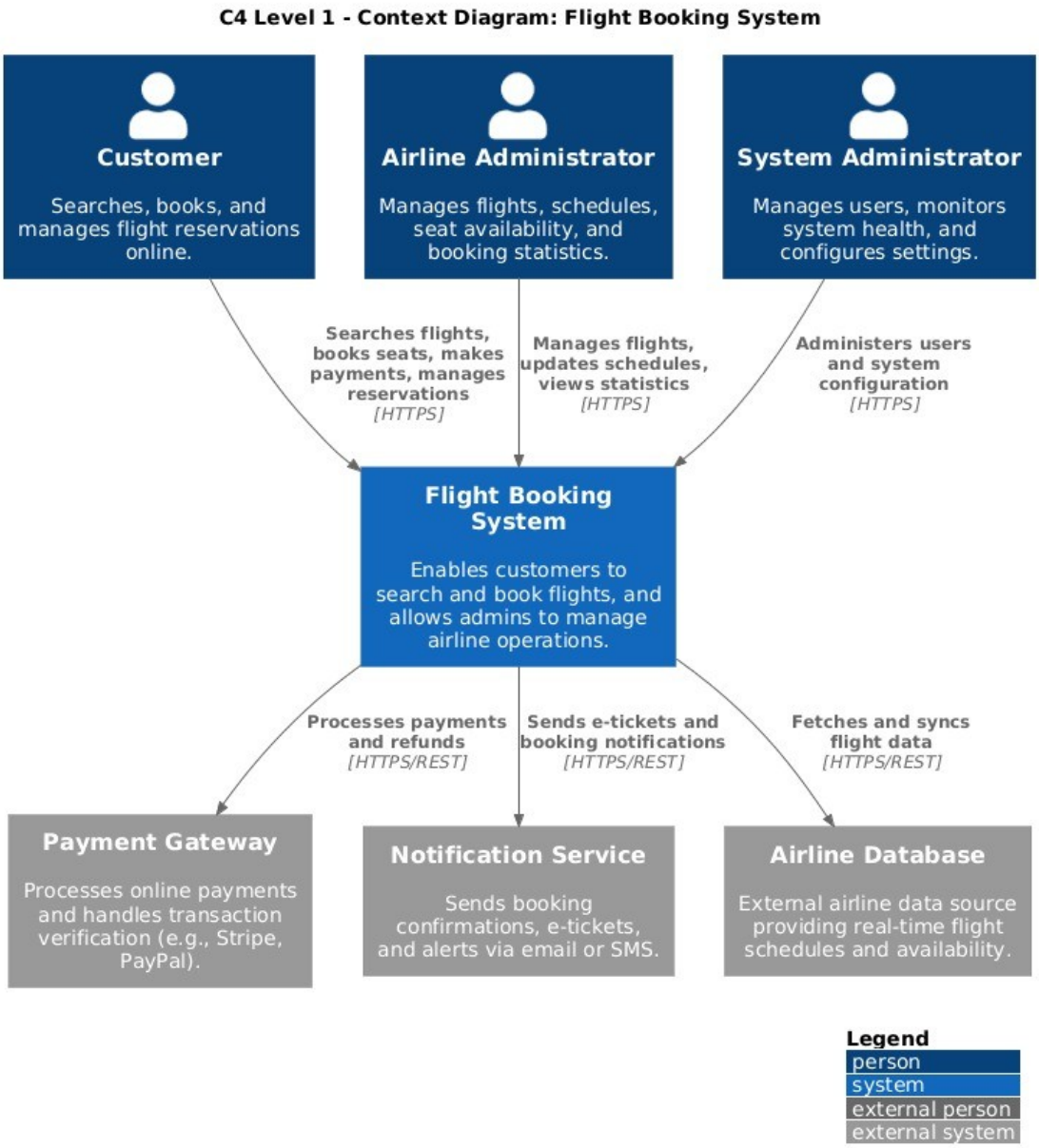

Figure 1: C4 Level 1 Context Diagram

Explanation: The customer searches flights, books seats, makes payments, and manages reservations through the system. Airline administrators update schedules, manage seat availability, and view booking statistics. System administrators manage users, monitor system health, and configure settings. The system communicates with a payment gateway for transaction processing, a notification provider for e-tickets and confirmations, and an airline database for updated flight data.

3. C4 Model Level 2 - Container Diagram

The container diagram breaks the Flight Booking System into internal deployable units and shows how they communicate.

Container	Technology	Responsibility
Web Application	React.js	Customer-facing interface for flight search, seat selection, booking, and account actions
Admin Portal	React.js	Interface for airline and system administrators
API Server	Node.js / REST API	Central business logic for authentication, flight search, booking, payment orchestration, and

		user management
Database	PostgreSQL	Persistent storage for users, flights, seats, bookings, payments, and history
Notification Service	Node.js / Mailer	Generates and sends booking confirmations and e-tickets
Airline Data Sync Service	Python / Scheduler	Periodically syncs schedules and availability from external airline databases

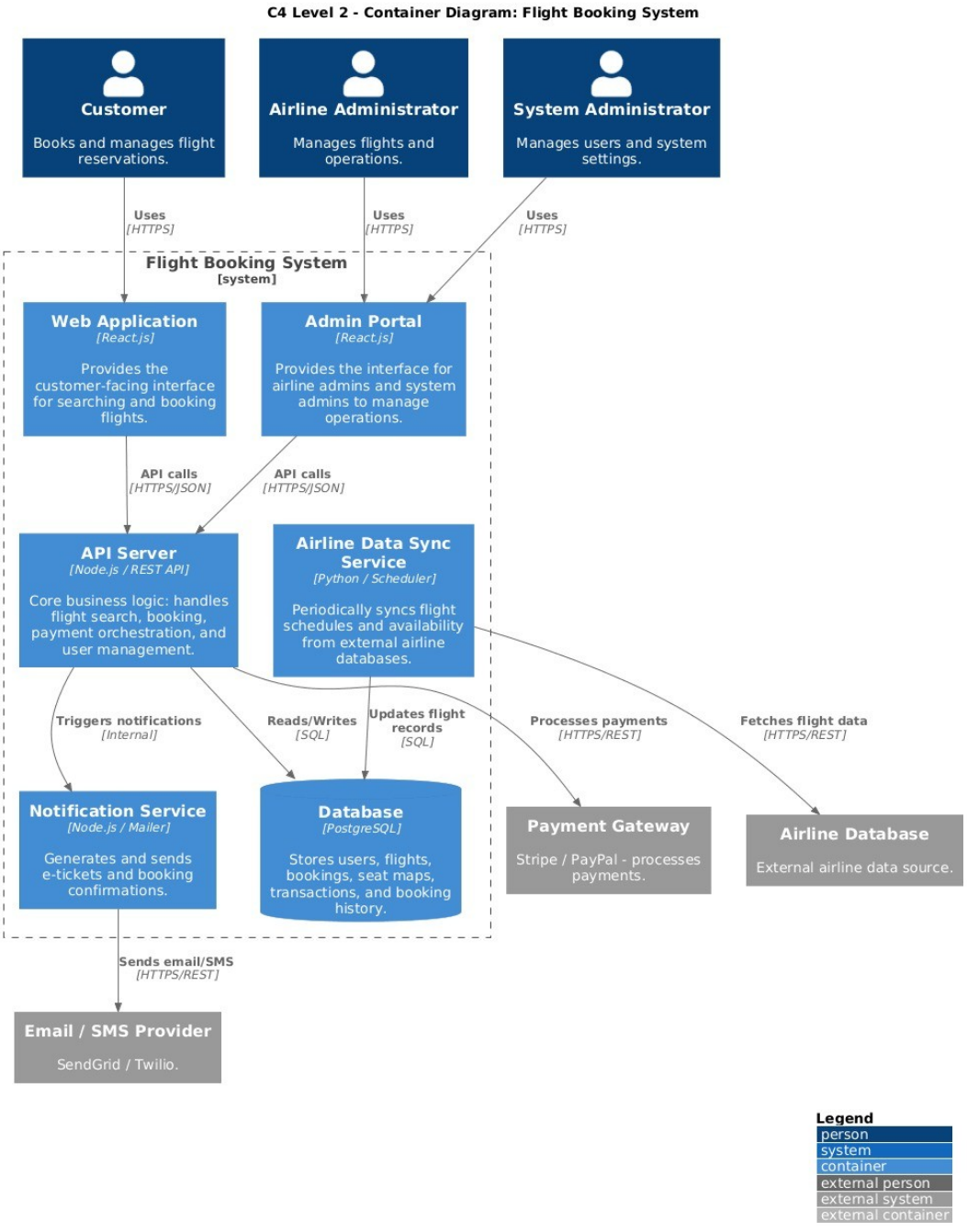

Figure 2: C4 Level 2 Container Diagram

Explanation: The Web Application and Admin Portal communicate with the API Server through HTTPS/JSON requests. The API Server handles most business rules and reads/writes data from the PostgreSQL database. The Airline Data Sync Service keeps flight records updated from external airline sources. The Notification Service sends messages through the external Email/SMS Provider. The Payment Gateway is used when customers complete payment.

4. Activity Diagram with Swimlanes

The activity diagram models the end-to-end flight booking process across three swimlanes: Customer, Flight Booking System, and Payment Gateway.

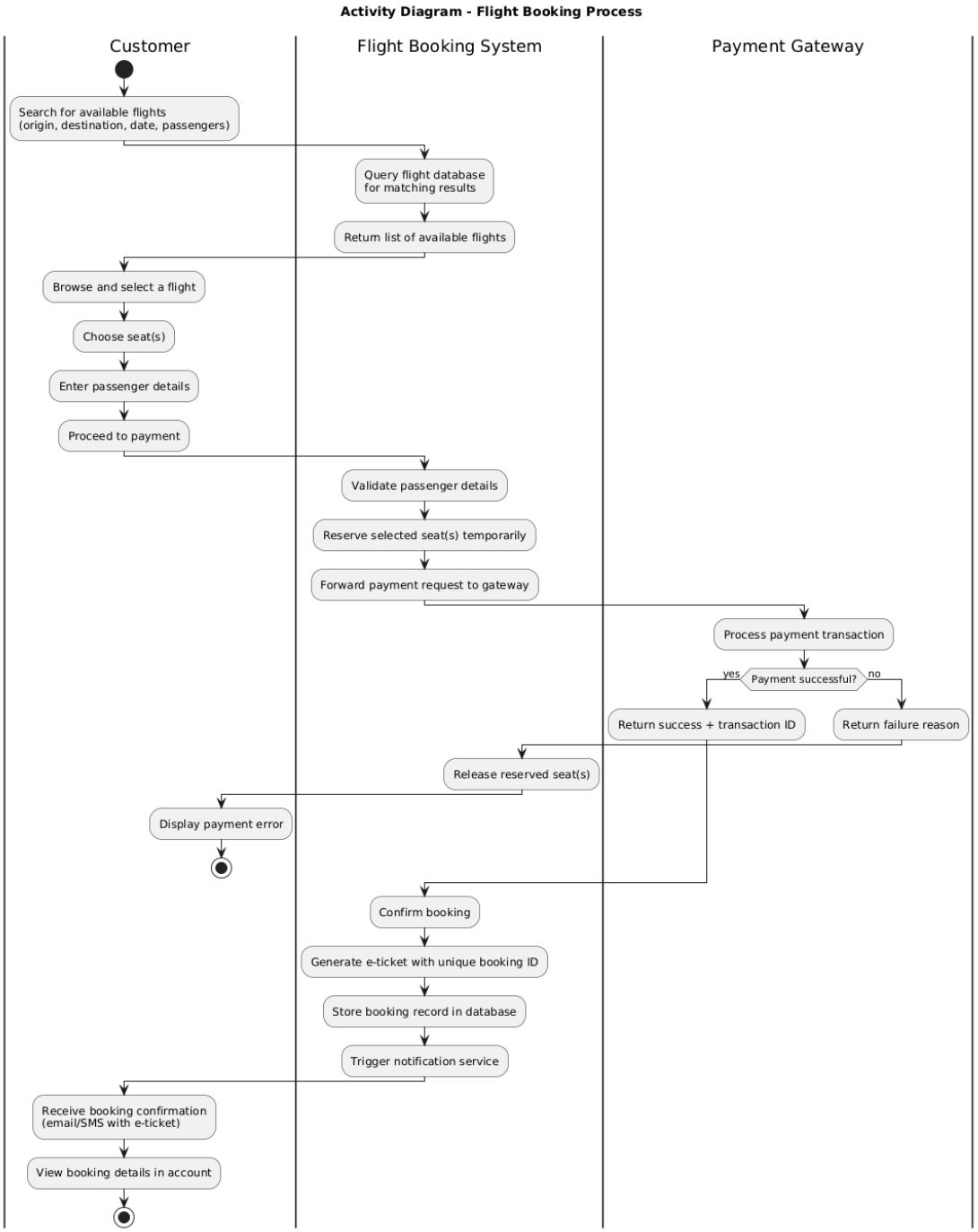

Figure 3: Activity Diagram with Swimlanes

Explanation: The customer begins by searching for flights and choosing a suitable flight and seat. The system validates passenger details, temporarily reserves selected seats, and forwards the payment request to the payment gateway. If payment fails, the system releases the reserved seats and displays an error. If payment succeeds, the system confirms the booking, generates an e-ticket, stores the booking record, and triggers the notification service.

5. Use Case Diagram

The use case diagram identifies the main services offered by the Flight Booking System and the actors that use them.

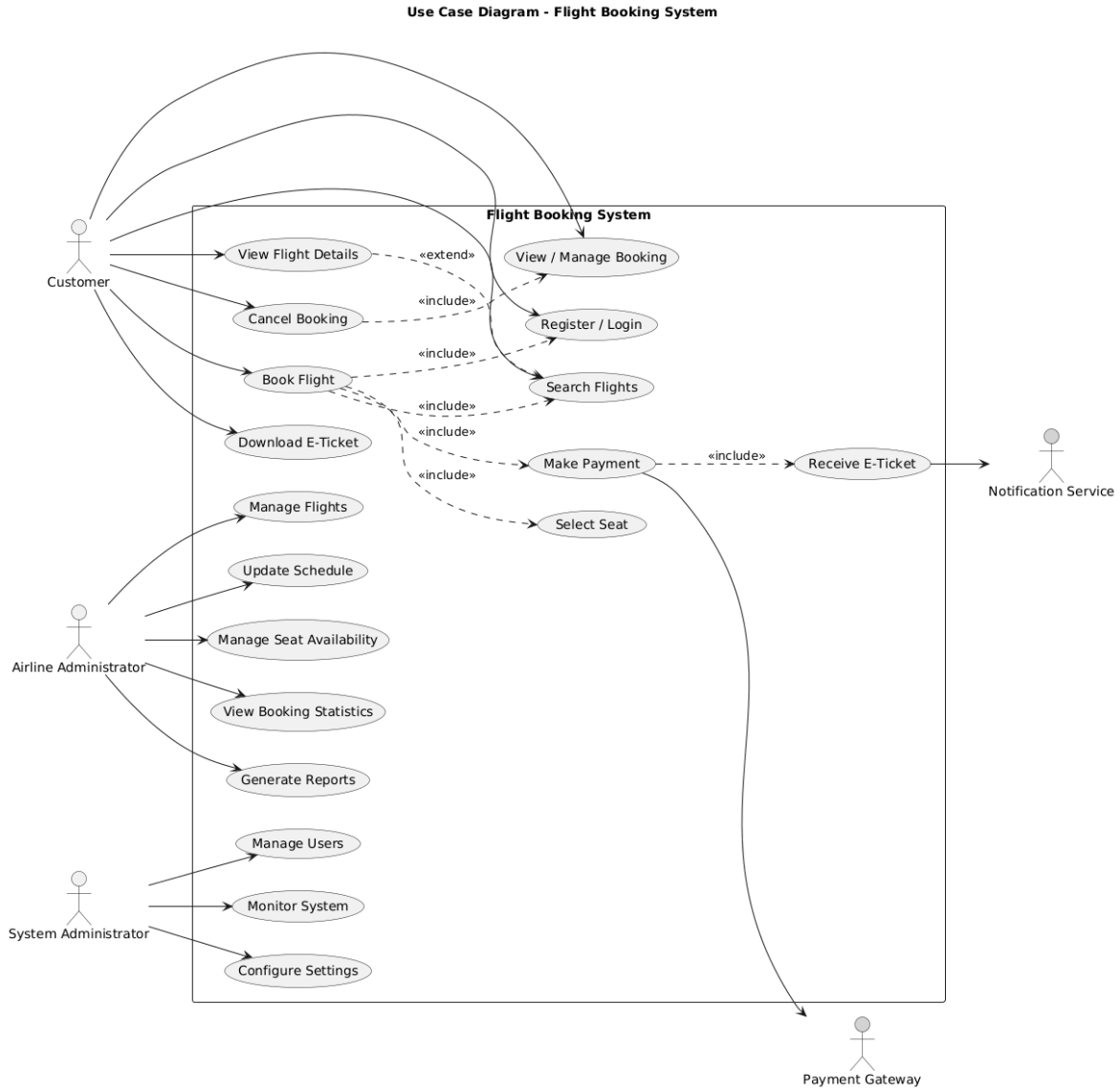

Figure 4: Use Case Diagram

Explanation: The Customer can search flights, view flight details, book a flight, select seats, make payment, receive/download an e-ticket, manage bookings, and cancel bookings. The Airline Administrator can manage flights, update schedules, manage seat availability, view booking statistics, and generate reports. The System Administrator can manage users, monitor the system, and configure settings. External actors such as the Payment Gateway and Notification Service support payment and ticket delivery flows.

6. Use Case Descriptions

UC-01: Search Flights

Field	Description
Use Case ID	UC-01
Name	Search Flights
Actor	Customer
Precondition	Customer is on the search page. Login is not required.
Trigger	Customer submits origin, destination, travel date, and number of passengers.
Main Flow	1. Customer enters search parameters. 2. System validates the inputs. 3. System queries available flights. 4. System returns matching flights with prices and durations. 5. Customer browses results.
Alternative Flow	If no flights are found, the system displays a no-

	available-flights message.
Postcondition	Available flight results are displayed.

UC-02: Book Flight

Field	Description
Use Case ID	UC-02
Name	Book Flight
Actor	Customer
Precondition	Customer is logged in and selected a flight.
Trigger	Customer clicks Book on a selected flight.
Main Flow	1. System displays seat map. 2. Customer selects available seat(s). 3. Customer enters passenger details. 4. System validates information and calculates total price. 5. Customer confirms booking summary. 6. System temporarily reserves seats. 7. System redirects customer to payment.
Alternative Flow	If a selected seat becomes unavailable, the system asks the customer to choose another seat.
Postcondition	Seats are temporarily reserved and the customer proceeds to payment.
Includes	UC-01 Search Flights, UC-03 Make Payment

UC-03: Make Payment

Field	Description
Use Case ID	UC-03
Name	Make Payment
Actor	Customer, Payment Gateway
Precondition	Booking was initiated and seats are temporarily reserved.
Trigger	Customer submits payment details.
Main Flow	1. Customer enters payment details. 2. System forwards payment request to Payment Gateway. 3. Payment Gateway processes transaction. 4. Payment Gateway returns success and transaction ID. 5. System confirms booking and generates e-ticket.
Alternative Flow	If payment fails, the system releases reserved seats and allows retry.
Postcondition	Payment is confirmed, booking is created, and e-ticket is generated.

UC-04: Cancel Booking

Field	Description
Use Case ID	UC-04
Name	Cancel Booking
Actor	Customer
Precondition	Customer is logged in and has a confirmed booking.
Trigger	Customer selects a booking and chooses Cancel.
Main Flow	1. Customer opens My Bookings. 2. Customer selects booking. 3. System displays cancellation policy and refund amount. 4. Customer confirms cancellation. 5. System cancels booking and releases seats. 6. System initiates refund if applicable. 7. System sends cancellation confirmation.
Alternative Flow	If the booking is non-refundable, the system informs the customer before confirmation.
Postcondition	Booking is cancelled, seats are available, and refund is initiated if allowed.

UC-05: Manage Flights

Field	Description
Use Case ID	UC-05
Name	Manage Flights
Actor	Airline Administrator
Precondition	Airline Administrator is logged in to the Admin Portal.
Trigger	Admin opens the flight management section.
Main Flow	1. Admin views flights. 2. Admin adds, edits, or deactivates a flight. 3. System validates flight information. 4. System saves changes. 5. Updated flight information appears to customers.
Alternative Flow	If a flight has existing bookings, the system warns the admin before major changes.
Postcondition	Flight records and availability are updated.

UC-06: Manage Seat Availability

Field	Description
Use Case ID	UC-06
Name	Manage Seat Availability
Actor	Airline Administrator
Precondition	Admin is logged in and has permission for the airline.
Trigger	Admin opens the seat management page for a flight.
Main Flow	1. Admin selects flight. 2. System displays seat map and status. 3. Admin updates seat availability or class. 4. System validates update. 5. System saves new seat status.
Alternative Flow	If a seat is already booked, the system prevents direct availability changes.
Postcondition	Seat availability is updated in the database.

UC-07: Manage Users

Field	Description
Use Case ID	UC-07
Name	Manage Users
Actor	System Administrator
Precondition	System Administrator is logged in.
Trigger	Admin opens user management page.
Main Flow	1. Admin searches users. 2. Admin adds, edits, suspends, or activates accounts. 3. System validates permissions. 4. System saves user updates.
Alternative Flow	If the admin lacks permission, the action is rejected.
Postcondition	User records are updated.

UC-08: Receive E-Ticket

Field	Description
Use Case ID	UC-08
Name	Receive E-Ticket
Actor	Customer, Notification Service
Precondition	Booking and payment are confirmed.
Trigger	System creates the confirmed booking record.
Main Flow	1. System generates unique ticket number. 2. System creates e-ticket. 3. Notification Service sends confirmation email/SMS. 4. Customer receives ticket and can download it.
Alternative Flow	If notification delivery fails, the system stores the ticket and allows download from account.

Postcondition	Customer has access to the e-ticket.

7. Sequence Diagrams

7.1 High-Level Sequence Diagram - Stakeholder View

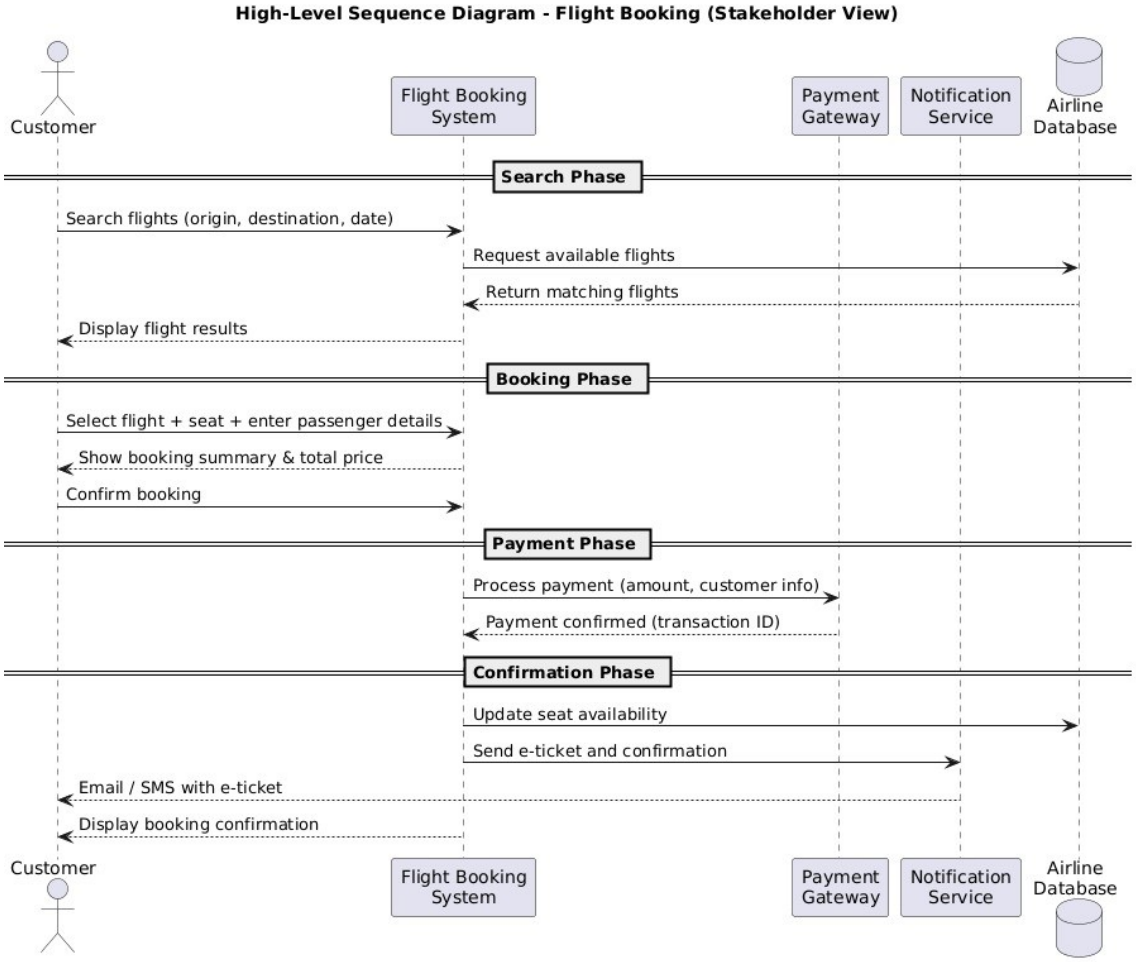

Figure 5: High-Level Sequence Diagram

Explanation: This sequence diagram focuses on major stakeholder interactions. It separates the booking scenario into search, booking, payment, and confirmation phases. It shows how the customer interacts with the Flight Booking System, how the system requests available flights from the airline database, how payment is processed through the payment gateway, and how confirmation is delivered through the notification service.

7.2 Detailed Sequence Diagram - Developer View

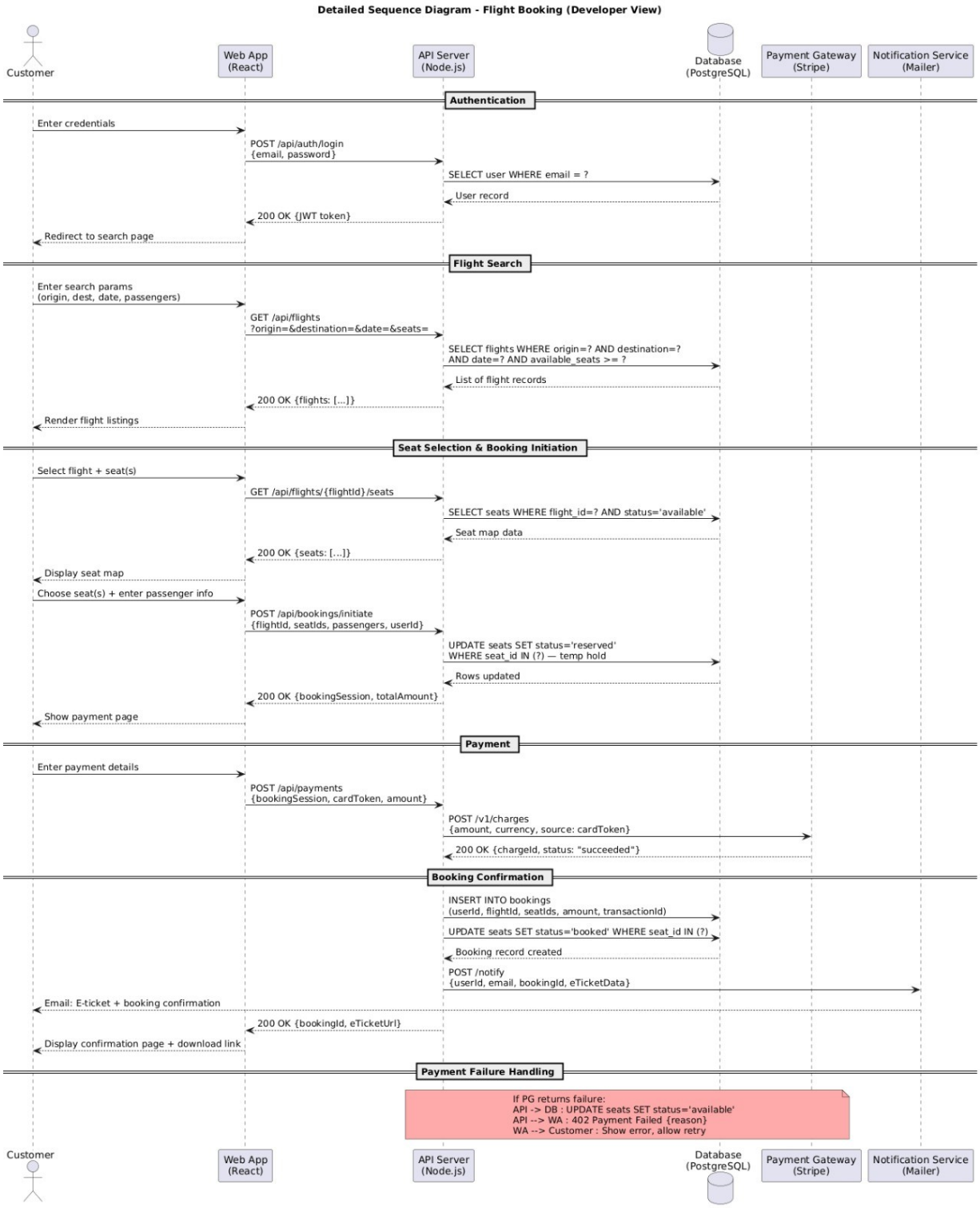

Figure 6: Detailed Sequence Diagram

Explanation: The detailed sequence diagram shows technical calls between the Web App, API Server, Database, Payment Gateway, and Notification Service. It includes authentication, REST API endpoints, database queries, seat reservation, payment processing, booking confirmation, e-ticket delivery, and payment failure handling.

8. Class Diagram

The class diagram models the static structure of the Flight Booking System. It includes attributes, operations, associations, inheritance, aggregation, and composition.

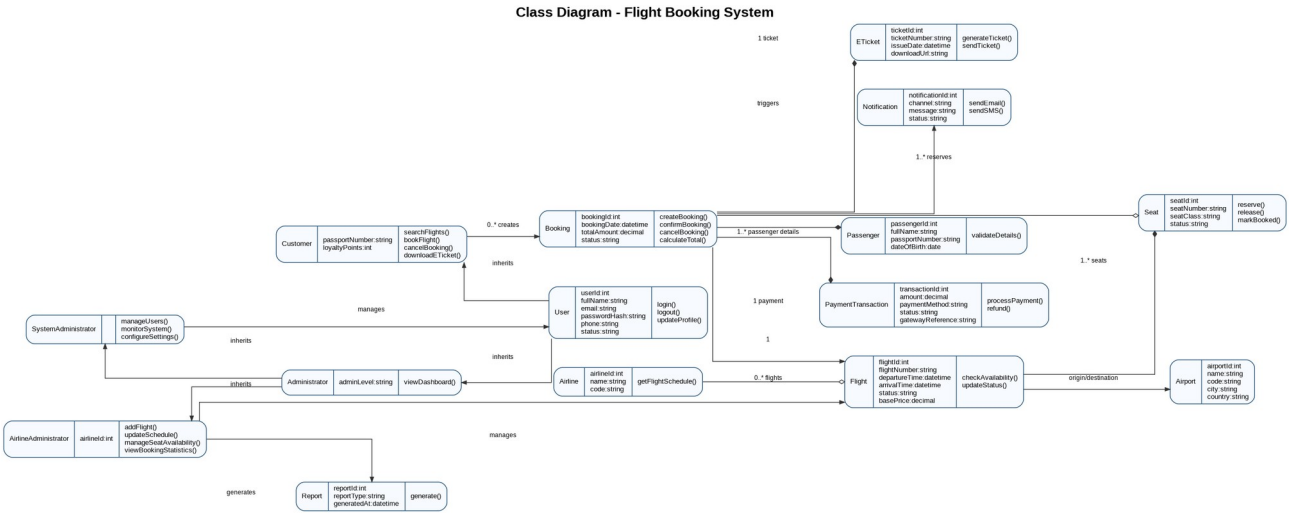

Figure 7: Class Diagram

Explanation: The main parent class is User, with Customer and Administrator as specialized user types. AirlineAdministrator and SystemAdministrator inherit from Administrator. The Flight class contains multiple Seat objects using composition because seats belong to a flight. The Booking class is connected to a customer, flight, selected seats, passenger details, payment transaction, and e-ticket. Passenger details, payment transaction, and e-ticket are modeled as composition parts of a booking because they are created as part of the booking process.

9. DFD Level 1 - Data Flow Diagram

Because the Flight Booking System is database-driven, a DFD is used as the representative behavior model. It shows how data moves between external entities, system processes, and internal data stores.

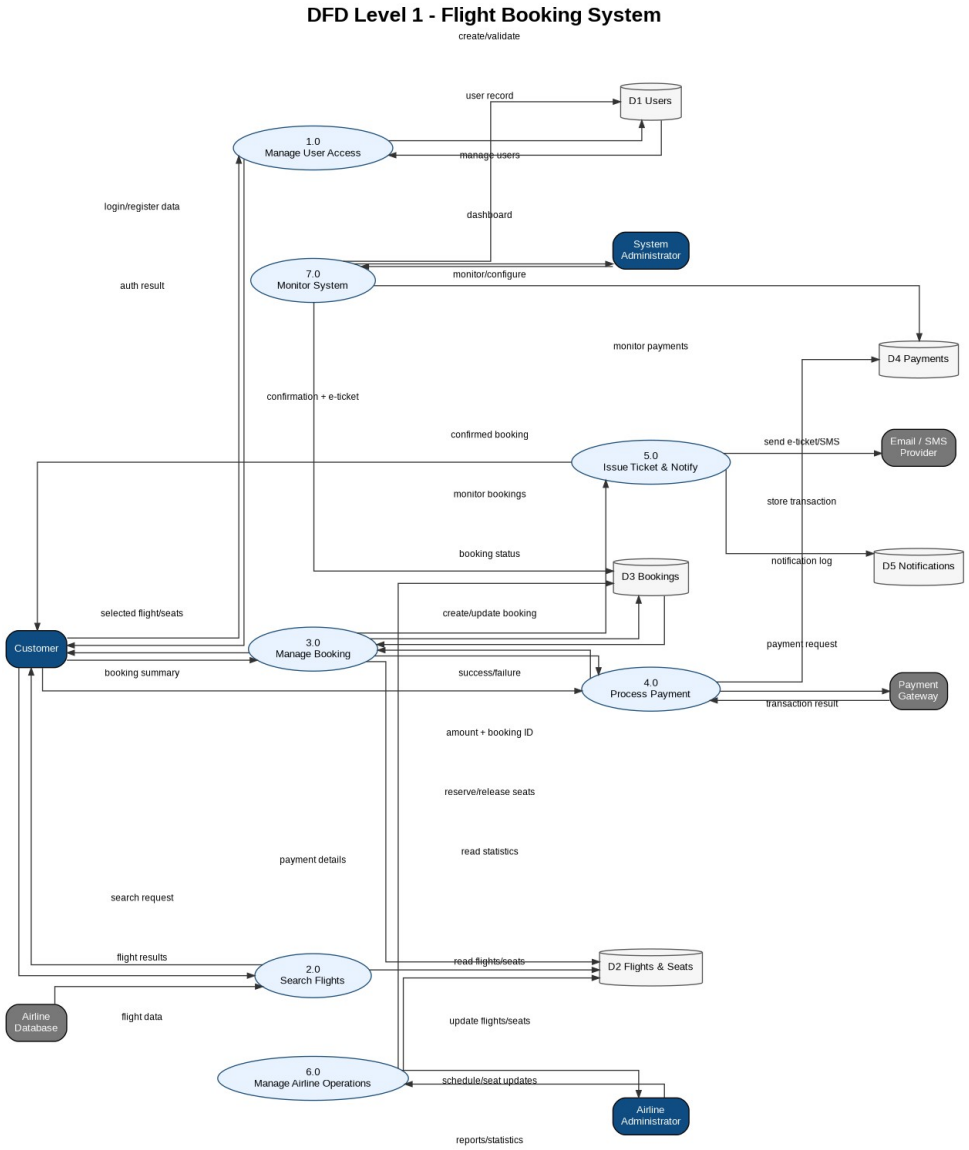

Figure 8: DFD Level 1

Explanation: The DFD divides the system into seven major processes: managing user access, searching flights, managing bookings, processing payment, issuing tickets and notifications, managing airline operations, and monitoring the system. The main data stores are users, flights and seats, bookings, payments, and notifications. External systems provide payment processing, email/SMS delivery, and airline flight data.

10. Final Notes and Assumptions

The system is modeled as a web-based platform with REST API communication between the front-end applications and the API server. The database is assumed to store all major operational data, including users, flights, seats, bookings, payments, notifications, and e-tickets. Payment details are sent securely to the payment gateway, while the local system stores only transaction references and payment status. Airline data is assumed to be synchronized periodically from external airline databases.

The final repository contains the required Markdown report, PDF report, PlantUML source files, and diagram images under the required folder structure.